

Programming — Föreläsning 1

Grundläggande programmering (i Python)

Marcell Ziegler

2026-06-02

Vad är programmering?

- Författandet av *algoritmer*,
- och implementerandet av dessa algoritmer m.h.a. ett programmeringsspråk så att datorn kan utföra dem.

Det är avsevärt viktigare att förstå
algoritmer än att förstå ett givet
programmeringsspråk!

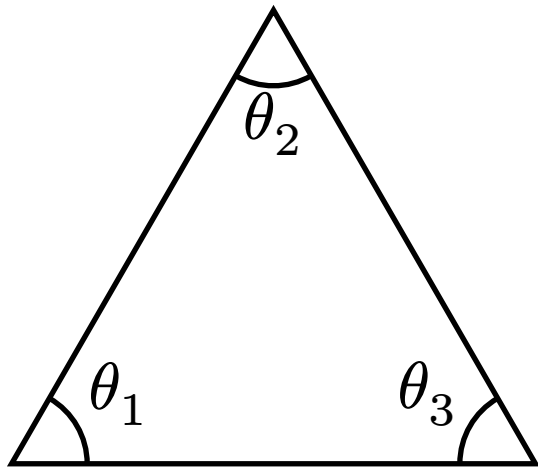
Att författa algoritmer

algorithm, algortimen, algoritmer (subst.)

*instruktionsföljd för lösning av prblem av (mer el. mindre)
matematiskt slag.*

— Svenska Akademiens Ordlista (2026)

Ett inledande exempel



Figur 1: En liksidig triangel

Som känt gäller:

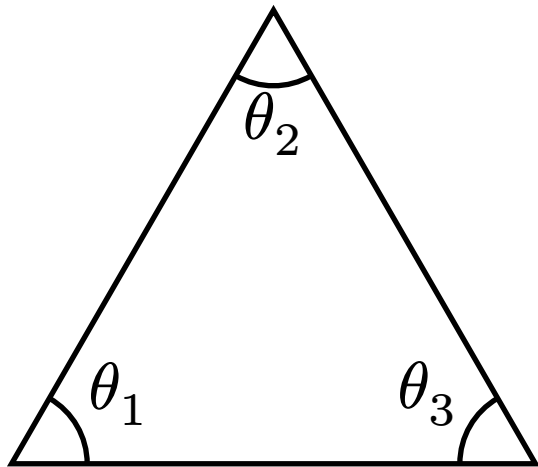
$$\theta_1 + \theta_2 + \theta_3 = 180^\circ$$

$$\theta_3 = 180^\circ - \theta_1 - \theta_2.$$

Vår algoritm blir då:

Att författa algoritmer

Ett inledande exempel



Figur 1: En liksidig triangel

Som känt gäller:

$$\theta_1 + \theta_2 + \theta_3 = 180^\circ$$

$$\theta_3 = 180^\circ - \theta_1 - \theta_2.$$

Vår algoritm blir då:

- 1 Subtrahera θ_1 från 180°
- 2 Subtrahera θ_2 från resultatet,
och

3 det du har kvar är θ_3 .

Ett lite mer invecklat exempel

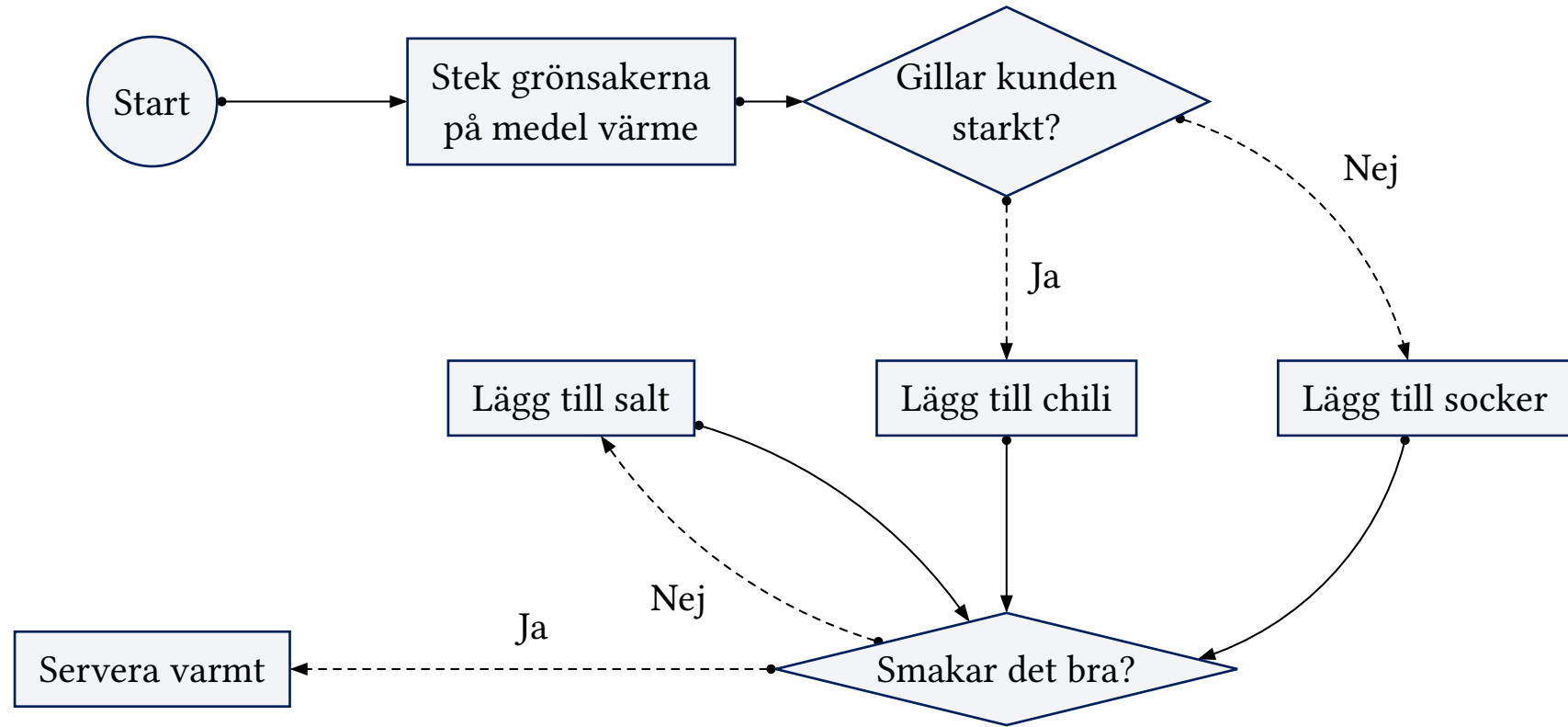
- 1 Stek grönsaker på medel värme.
- 2 **om** kunden gillar starkt **gör**
- 3 └ Lägg till chili.
- 4 **annars**
- 5 └ Lägg till socker.
- 6 **medan** det inte smakar bra
- 7 └ Lägg till salt.
- 8 Servera varmt.



Figur 2: "Stir Fry Wok - Free For Commercial Use - FFCU" by Free for Commercial Use är licenserad under CC BY-SA 2.0.

Att författa algoritmer

Ett flödesschema



Figur 3: Flödesschema för matlagningsalgoritmen

Hur algoritmer blir till kod

Hur algoritmer blir till kod

Kompilering

Hur algoritmer blir till kod

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello,
5     World!\n";
6     return 0;
7 }
```

C++



GCC Intermediate Representation

```
1 # 1 "hello.cpp"
2 # 1 "<built-in>"
3 # 1 "<command-line>"
4 # 2 "/usr/include/stdc-predef.h" 1 3 4
5 # 1 "<command-line>" 2
6 # 2 "hello.cpp"
7 # 2 "/usr/include/c++/15.2.1/iostream" 1 3
8 # 41 "/usr/include/c++/15.2.1/iostream" 3
9 # 2 "/usr/include/c++/15.2.1/bits/requires_hosted.h" 1 3
10 # 32 "/usr/include/c++/15.2.1/bits/requires_hosted.h" 3
11 # 2 "/usr/include/c++/15.2.1/x86_64-linux-gnu/bits/c++config.h" 1 3
12 # 38 "/usr/include/c++/15.2.1/x86_64-linux-gnu/bits/c++config.h" 3
13 #pragma GCC diagnostic push
14 #pragma GCC diagnostic ignored "-Wvariadic-macros"
15
16 #pragma GCC diagnostic ignored "-Wc++13-extensions"
17 #pragma GCC diagnostic ignored "-Wc++24-extensions"
18 # 337 "/usr/include/c++/15.2.1/x86_64-linux-gnu/bits/c++config.h" 3
19 namespace std
20 {
21
22
23 ...
24
25
26
27
28
29 extern wistream wcin;
30 extern wostream wcout;
31 extern wostream wcerr;
32 extern wostream wlog;
33 # 84 "/usr/include/c++/15.2.1/iostream" 3
34 __extension__ __asm__ (".globl _ZSt21ios_base_library_initv");
35
36
37
38 }
39 # 2 "hello.cpp" 2
40
41
42 # 3 "hello.cpp"
43 int main() {
44     std::cout << "Hello, World!\n";
45     return 0;
46 }
```

Hur algoritmer blir till kod

Kompilering

Hur algoritmer blir till kod

```
1 # 1 "hello.cpp"
2 # 1 "<built-in>"
3 # 1 "<command-line>"
4 # 2 "/usr/include/stdc-predef.h" 1 3 4
5 # 1 "<command-line>" 2
6 # 2 "hello.cpp"
7 # 2 "/usr/include/c++/15.2.1/iostream" 1 3
8 # 41 "/usr/include/c++/15.2.1/iostream" 3
9 # 2 "/usr/include/c++/15.2.1/bits/requires_hosted.h" 1 3
10 # 32 "/usr/include/c++/15.2.1/bits/requires_hosted.h" 3
11 # 2 "/usr/include/c++/15.2.1/x86_64-pc-linux-gnu/bits/c++config.h" 1 3
12 # 38 "/usr/include/c++/15.2.1/x86_64-pc-linux-gnu/bits/c++config.h" 3
13 #pragma GCC diagnostic push
14 #pragma GCC diagnostic ignored "-Wvariadic-macros"
15
16 #pragma GCC diagnostic ignored "-Wc++13-extensions"
17 #pragma GCC diagnostic ignored "-Wc++24-extensions"
18 # 337 "/usr/include/c++/15.2.1/x86_64-pc-linux-gnu/bits/c++config.h" 3
19 namespace std
20 {
21
22
23
24 ...
25
26
27
28
29 extern wistream wcin;
30 extern wostream wcout;
31 extern wostream wcerr;
32 extern wostream wclog;
33 # 84 "/usr/include/c++/15.2.1/iostream" 3
34 __extension__ __asm__ ("global _ZSt2lios_base_library_initv");
35
36
37 }
38
39 # 2 "hello.cpp" 2
40
41
42 # 3 "hello.cpp"
43 int main() {
44     std::cout << "Hello, World!\n";
45     return 0;
46 }
```

GCC Intermeideate Representation



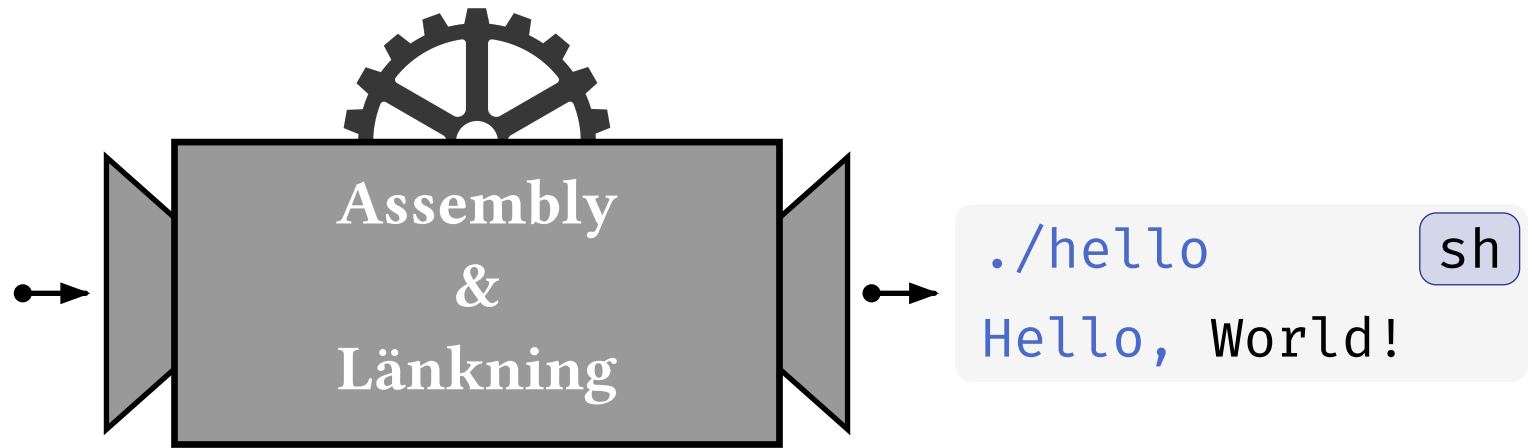
```
1 .file "hello.cpp"
2 #APP
3 .globl _ZSt2lios_base_library_initv
4 .section .rodata
5 .text
6 .LC0:
7 .string "Hello, World!\n"
8 #NO_APP
9 .text
10 .globl main
11 .type main, @function
12 main:
13 .LFB1976:
14 .cfi_startproc
15 pushq %rbp
16 .cfi_def_cfa_offset 16
17 .cfi_offset 7, -16
18 movq %rsp, %rbp
19 .cfi_def_cfa_register 6
20 leaq .LC0(%rip), %rdx
21 leaq _ZSt4cout(%rip), %rax
22 movq %rdx, %rsi
23 movq %rax, %rdi
24 call _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@
25 movl $0, %eax
26 popq %rbp
27 .cfi_def_cfa 7, 8
28 ret
29 .cfi_endproc
30 .LFE1976:
31 .size main, .-main
32 .ident "GCC: (GNU) 15.2.1 20260209"
33 .section .note.GNU-stack,"",@progbits
```

x86_64 Assembly

Hur algoritmer blir till kod

Kompilering

```
1  .file "hello.cpp"
2  #APP
3  .globl _ZSt21ios_base_library_initv
4  .section .rodata
5  .text
6  .LC0:
7  .string "Hello, World!\n"
8  #NO_APP
9  .text
10 .globl main
11 .type main, @function
12 main:
13 .LFB1976:
14 .cfi_startproc
15 pushq %rbp
16 .cfi_def_cfa_offset 16
17 .cfi_offset 7, -16
18 movq %rsp, %rbp
19 .cfi_def_cfa_register 6
20 leaq .LC0(%rip), %rdx
21 leaq _ZSt4cout(%rip), %rax
22 movq %rdx, %rsi
23 movq %rax, %rdi
24 call _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@
25 movl $0, %eax
26 popq %rbp
27 .cfi_def_cfa 7, 8
28 ret
29 .cfi_endproc
30 .LFE1976:
31 .size main, .-main
32 .ident "GCC: (GNU) 15.2.1 20260209"
33 .section .note.GNU-stack,"",@progbits
```



Kompilering

- *Kompilera* betyder ”sammaställa”

Kompilering

- *Kompilera* betyder ”sammaställa”
- Vi jobbar med hela filen samtidigt

Kompilering

- *Kompilera* betyder ”sammaställa”
- Vi jobbar med hela filen samtidigt
- Slutresultatet kan inte enkelt göras om till källkod

Kompilering

- *Kompilera* betyder ”sammaställa”
- Vi jobbar med hela filen samtidigt
- Slutresultatet kan inte enkelt göras om till källkod
- Snabbt och effektivt resultat

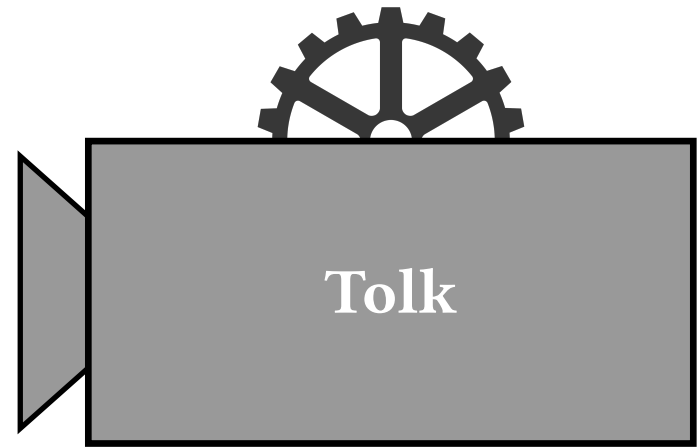
Kompilering

- *Kompilera* betyder ”sammaställa”
- Vi jobbar med hela filen samtidigt
- Slutresultatet kan inte enkelt göras om till källkod
- Snabbt och effektivt resultat
- Kan bara köras på den typ av dator som det är kompilerat för

Tolkning

```
1 def greet(name)
2     print(f"Hello,
3         {name}!")
4 greet("Marcell")
```

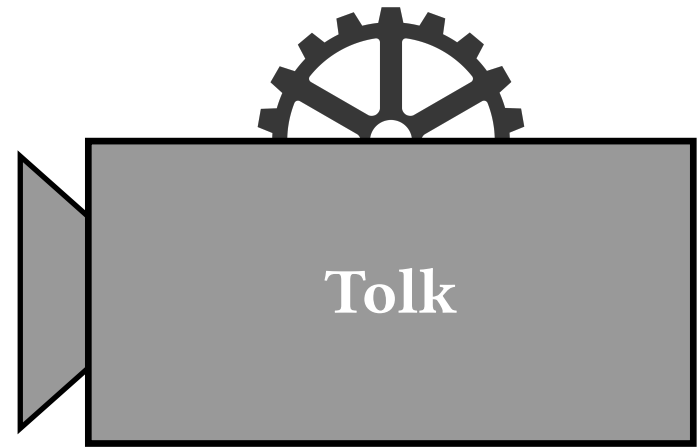
Python



Tolkning

```
1  def greet(name)
2      print(f"Hello, {name}!")
3
4  greet("Marcell")
```

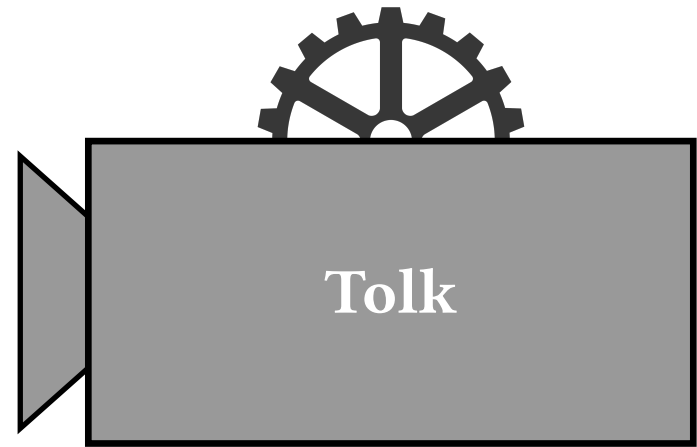
Python



Tolkning

```
1 def greet(name)
2     print(f"Hello,
3         {name}!")
4 greet("Marcell")
```

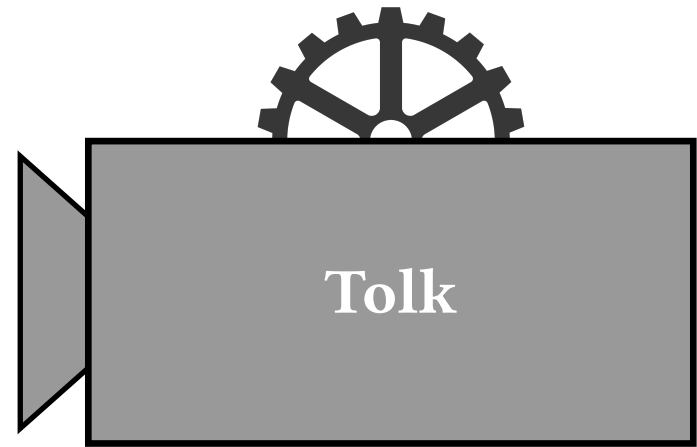
Python



Tolkning

```
1 def  
  greet(name)  
2     print(f"Hello,  
    {name}!")  
3  
4 greet("Marcell")
```

Python



Tolkning

- Portabelt, koden funker överallt där det finns en tolk

Tolkning

- Portabelt, koden funker överallt där det finns en tolk
- Alla målmaskiner måste kunna köra tolken

Tolkning

- Portabelt, koden funker överallt där det finns en tolk
- Alla målmaskiner måste kunna köra tolken
- Körbar kod kan distribueras i läsligt format

Tolkning

- Portabelt, koden funker överallt där det finns en tolk
- Alla målmaskiner måste kunna köra tolken
- Körbar kod kan distribueras i läsligt format
- Ofta lägre prestanda

Terminalen

Terminalen

- Din dators kommandotolk

Terminalen

- Din dators kommandotolk
- Ett sätt att interagera med datorer utan GUI

Terminalen

- Din dators kommandotolk
- Ett sätt att interagera med datorer utan GUI
- Här körs de flesta program för programmering

Terminaldemo!

Hur man skriver och kör kod

Hello, World!

1. Öppna VSCode
2. Öppna en mapp
3. Skriv in koden nedan i en fil

```
1 print("Hello, World!")
```

Python

4. Kör koden!

Rast! (15 min)

Variabler

Vad är en variabel?

- Som en låda för data med...
 - ...ett namn
 - ...ett värde

Vad är en variabel?

- Som en låda för data med...
 - ...ett namn
 - ...ett värde
- Variabler sparas inom sitt **scope**, eller tills programmet är slut.

Hur du använder variabler

- En variabel skapar du som

```
1 variabelns_namn = ...
```

Python

Hur du använder variabler

- En variabel skapar du som

```
1 variabelns_namn = ...
```

Python

- Variabeln kan heta vad som helst

Hur du använder variabler

- En variabel skapar du som

```
1 variabelns_namn = ...
```

Python

- Variabeln kan heta vad som helst
 - Engelska namn rekommenderas
 - Alltid `snake_case`

Hur du använder variabler

- En variabel skapar du som

```
1 variabelns_namn = ...
```

Python

- Variabeln kan heta vad som helst
 - Engelska namn rekommenderas
 - Alltid `snake_case`
- Värdet kan vara vad som helst, Python är **dynamiskt typat**.

Datatyper

Heltal

- Datorer är binära: talen lagras i bas 2
- Annars helt normala

2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	0	1	0	1	0	1	1	0	0	0	1	1	0	1

Heltal

- Datorer är binära: talen lagras i bas 2
- Annars helt normala

$$\begin{array}{cccccccccccccccccc}
 2^{15} & 2^{14} & 2^{13} & 2^{12} & 2^{11} & 2^{10} & 2^9 & 2^8 & & 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\
 2^{15} + 0 + 0 + 2^{12} + 0 + 2^{10} + 0 + 2^8 & + 2^7 + 0 + 0 + 0 + 2^3 + 2^2 + 0 + 1
 \end{array}$$

Heltal

- Datorer är binära: talen lagras i bas 2
- Annars helt normala

$$\begin{array}{cccccccccccccccccc}
 2^{15} & 2^{14} & 2^{13} & 2^{12} & 2^{11} & 2^{10} & 2^9 & 2^8 & & 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\
 2^{15} + 0 + 0 + 2^{12} + 0 + 2^{10} + 0 + 2^8 & + 2^7 + 0 + 0 + 0 + 2^3 + 2^2 + 0 + 1 \\
 \\
 = 32\,768 + 4096 + 1024 + 256 + 128 + 8 + 4 + 1 = 38\,285
 \end{array}$$

Flyttal (decimaltal)

En separat **mantissa** och **exponent**:

$$m \cdot 2^x$$

där m är mantissan och x är exponenten.

Notera 2:an, då vi är i bas 2. I stället för **tio**potensform har vi **två**potensform.

Precisionen är (ofta) begränsad till 23 binära värdesiffror.

Notation för flyttal

Samma som på miniräknare:

100.2

13.281

1e3 (10^3)

4219.345e-3 ($4219.345 \cdot 10^{-3}$)

De skrivs alltså i bas 10, men lagras i bas 2 som flyttal.

Booleska värden

- Också känt som sant/falskt-värden

Booleska värden

- Också känt som sant/falskt-värden
- Skrivs med nyckelorden `True` / `False` (OBS! Stor bokstav)

Booleska värden

- Också känt som sant/falskt-värden
- Skrivs med nyckelorden `True` / `False` (OBS! Stor bokstav)
- Lagras som en 1:a för `True`, och 0:a för `False`
 - Det betyder dock inte att de är likvärdiga med en `int`!

Text

- De är egentligen listor med karaktärer. (**grapheme clusters**)
- Dessa lagras kodade, som ett schiffer, med 8 binära siffror för varje karaktär.

Text

- De är egentligen listor med karaktärer. (**grapheme clusters**)
- Dessa lagras kodade, som ett schiffer, med 8 binära siffror för varje karaktär.
- De vanligaste kodningarna är `UTF-8` (Unicode) och `ASCII`.

Hur datatyper interagerar

- Du kan inte kombinera datatyper hur som helst.

Hur datatyper interagerar

- Du kan inte kombinera datatyper hur som helst.
- Ex. är `tal1 + tal2` en giltig operation, men `text + tal` är inte det.
 - Läs vidare [i boken!](#)

Räkna i Python

Binära operationer

Med en **binär operator** kan du utföra en **binär operation**. Dessa är:

- Addition
- Subtraktion
- Division
- Multiplikation
- Modulus

De är *binära* eftersom de krävs två **operander**.

Binära operatörer

Operator	Namn	Användning
<code>+</code>	Addition	<code>5 + 5</code>
<code>-</code>	Subtraktion	<code>3 - 4</code>
<code>/</code>	Division	<code>1.5 / 3</code>
<code>*</code>	Multiplikation	<code>7.6 * 5</code>
<code>**</code>	Exponentiering	<code>3**2</code>
<code>//</code>	Trunkerande division	<code>3.6 // 2</code>
<code>%</code>	Modulus	<code>6 % 2</code>

Att räkna med variabler

Precis som i matten kan du räkna med värdena som finns i variabler¹.

Du kan också ändra variablers värde efter du skapat dem:

```
1 num = 3
2 num += 2
3 num -= 1
4 num /= 2
5 num *= 4
```

python

¹Mer intressant matte finns [i boken!](#)

Vad är värdet av `num` nu?

In- och utdata i program

Funktionen `print()`

Hur man skriver ut text är lite olika i olika språk, men i Python använder ni `print(<det som skall skrivas ut>)`¹.

¹Kom ihåg `print("Hello, World!")` från innan.

Funktionen `print()`

Hur man skriver ut text är lite olika i olika språk, men i Python använder ni `print(<det som skall skrivas ut>)`¹.

Varför är `"Hello, World!"` inom citattecken?

¹Kom ihåg `print("Hello, World!")` från innan.

Funktionen `input()`

Denna funktion läser en och endast en rad från terminalen per anrop.

Resultatet returneras *alltid* som text (`str`) oavsett vad användaren skrivaren.

Detta används för att få in data som kan ändras vid varje körning.

Ett exempel med `input()`

```
1 namn = input("Vad är ditt namn: ")  
2  
3 print("Hej, " + namn)
```

Python

Vad ger detta ut på terminalen?

Att konvertera datatyper

Typkonvertering

Datatyper konverteras genom att skriva `typ(data)`. Koden kraschar om konverteringen är ogiltig.

Typkonvertering

Några exempel:

```
1 int("5")      -> 5
2 str(3.12)     -> "3.12"
3 float("2e7")  -> 2e6 = 2000000.0
4 bool("True")  -> True
5 bool(1)       -> True
6 int(False)    -> 0
7 int("Hej")    -> Krasch
```

Python

Detta är mest användbart för att hantera indata som kommer i `str`-format.

Att skriva ut icke-text

Funktionen `print()` klarar alla datatyper som kan göras till text, men tänk om du vill ha en beskrivning?

Med två värden: `student` & `points` kan vi t.ex. skriva

```
1 print("Eleven")
2 print(student)
3 print("har poäng:")
4 print(points)
```

Python

Att skriva ut icke-text

Föregående exempel är osmidigt och fult, vi använder i stället f-strängar.

```
1 print(f"Eleven {student} har poäng: {points}")
```

Python

Värdena inom klamrar utvärderas och konverteras automatiskt till `str`.

Att konvertera datatyper

Att skriva ut icke-text

Föregående exempel är osmidigt och fult, vi använder i stället f-strängar.

```
1 print(f"Eleven {student} har poäng: {points}")
```

Python

Värdena inom klamrar utvärderas och konverteras automatiskt till `str`.

Vi kan också använda strängaddition:

```
1 print("Eleven " + student + "har poäng: " +  
    str(points))
```

python

Att konvertera datatyper

Notera hur vi behövde konvertera manuellt här! Notera också mellanslagen som måste vara med!

Kontrollstrukturen

if-satsen

Om ____ är **True**, gör ____.

```
1 if expr:  
2     ...
```

Python

Viktigt att notera:

- Kolon efter uttrycket
- Indetering med 4 mellanslag
- **if**-satsen utgör ett **block** men inte ett **scope**

else-satsen

Om förra **if**-satsen inte kördes, gör ____.

```
1  if expr:  
2      ...  
3  else:  
4      ...
```

Python

Viktigt att notera:

- Samma indenteringsgrad som sin tillhörande **if**-sats
- Ingen blankrad mellan koden i **if**-blocket och **else**-blocket

- Utgör ett **block** men inte ett **scope**

elif-satsen

Om förra **if**-satsen inte kördes, men ____ är **True**, gör ____.

```
1  if expr:  
2      ...  
3  elif expr2:  
4      ...
```

Python

Viktigt att notera:

- Samma indenteringsgrad som sin tillhörande **if**-sats
- Ingen blankrad mellan koden i **if**-blocket och **elif**-blocket

- Utgör ett **block** men inte ett **scope**

Kedja **if**-satser

Man kan skapa en kedja av kontrollstrukturer:

```
1 if expr1:  
2     ...  
3 elif expr2:  
4     ...  
5 elif expr3:  
6     ...  
7 else:  
8     ...
```

Python

Uttrycken utvärderas i ordning, den första som är sann körs sen körs inga andra. Om ingen är sann körs alltid `else`-satsen.

Kedjor med **match**-satsen

En mer sällan använd sats är **match**-satsen. Den låter dig göra en mer kompakt kedja baserat på olika möjligheter på värdet av samma uttryck.

```
1 match expr:  
2     case val1:  
3         ...  
4     case val2:  
5         ...
```

Python

Den första gren vars värde överensstämmer med `expr` körs. Om ingen matchar, körs ingen. Endast en gren kan köras totalt. `case _:` hade matchat vad som helst.

Booleska uttryck (Sant/Falskt-uttryck)

Jämförelseoperatörer

Operator	Användning	Beskrivning
<code>=</code>	<code>a = b</code>	Likhet. <code>True</code> om a har samma värde som b, annars <code>False</code> .
<code>≠</code>	<code>a ≠ b</code>	Inte lika med. Omvänt av likhet.
<code>></code>	<code>a > b</code>	Större än. Fungerar främst för nummer av olika slag.
<code><</code>	<code>a < b</code>	Mindre än. Fungerar främst för nummer av olika slag.
<code>≥</code>	<code>a ≥ b</code>	Större än eller lika med. Fungerar främst för nummer av olika slag.
<code>≤</code>	<code>a ≤ b</code>	Mindre än eller lika med. Fungerar främst för nummer av olika slag.

Logiska operatörer

Operator	Användning	Beskrivning
<code>and</code>	<code>< ... > and < ... ></code>	Bir <code>True</code> om de booleska uttrycken på båda sidor är <code>True</code> . Annars är den <code>False</code> .
<code>or</code>	<code>< ... > or < ... ></code>	Blir <code>True</code> om höger eller vänster sida är <code>True</code> , eller om båda är <code>True</code> . Blir <code>False</code> om både höger och vänster är <code>False</code>

Operator	Användning	Beskrivning
<code>not</code>	<code>not <... ></code>	Byter sanningsvärde på det som kommer efter. <code>True</code> blir <code>False</code> , och vice versa.

Att använda booleska uttryck

Booleska variabler

```
1 is_true = True
2 if is_true:
3     ...
```

Python

Kombinerade uttryck

```
1 if (a == 5 and b > 5) or (c < 5 or d ≥ 7):
2     ...
```

Python

Matematiska intervall

```
1 if 5 ≤ x < 72:  
2     ...
```

Python

Speciella operatörer

`in`/`not in` kollar om värdet till vänster finns i kollektionen till höger.

`is`/`is not` kollar om ett värde till vänster har samma adress i minnet som värdet till höger.

Detta används sedvanligt för att kolla likhet, eller olikhet, med `None`

```
1 if var is None:  
2     ...  
3  
4 if var is not None:
```

Python

5

...